

AI MODEL EXPLORER

Case Study 1 — CI/CD Implementation using GitHub, Jenkins, Docker & Kubernetes

Final Implementation & Evidence Report

Field	Value
Project	AI Model Explorer
Repository	KarthikBalaje/ai-model-explorer
Working branch	feature/ai-model-explorer
Environment	Windows 11 • Docker Desktop • Jenkins • Minikube • Kubernetes
Jenkins Build	#8 — successful

Prepared: 30 September 2026

1. Executive Summary

The project implements an interactive Flask-based AI Model Explorer powered by live OpenRouter model metadata. Users can select a company, view company-specific model versions and dynamically derived model types, then inspect model ID, launch date, context length, architecture, pricing, modalities, capabilities and supported parameters.

The application was quality-checked with Black, Flake8 and Pytest, containerized using a multi-stage Dockerfile, scanned with Trivy, built and pushed by Jenkins, and deployed to Kubernetes through Minikube. A live Minikube application screenshot is included as submission evidence.

2. Assessment Coverage

Assessment Area	Weight	Evidence / Implementation	Status
Flask + APIs — app.py	20	Flask UI and API routes; 7 automated tests passed.	Verified
Git repository, GitFlow & branching	20	Public GitHub repository; main, staging and feature/ai-model-explorer branches used.	Implemented
Dockerfile	15	Multi-stage image; ~128.1 MiB; Trivy gate cleared.	Verified
Jenkinsfile	15	Jenkins Build #8 successful; checkout, quality checks, Docker build, Trivy and push.	Verified
Kubernetes deployment.yaml	10	2 replicas, RollingUpdate, probes, resources and Build #8 image.	Verified
Kubernetes service.yaml	10	NodePort service targeting port 5000; served through Minikube.	Verified
Application output screenshot	10	Live Minikube browser evidence included below.	Evidence included

The weights above are the Case Study 1 assessment categories; this report records evidence rather than assigning an independent score.

```

(.venv)
Balaje@Karthik MINGW64 /h/ai-model-explorer (feature/ai-model-explorer)
$ git branch -a
* feature/ai-model-explorer
  main
  remotes/origin/HEAD -> origin/main
  remotes/origin/feature/ai-model-explorer
  remotes/origin/main
  remotes/origin/staging
(.venv)
Balaje@Karthik MINGW64 /h/ai-model-explorer (feature/ai-model-explorer)
$ git log --oneline --decorate --graph --all -15
* a44307e (HEAD -> feature/ai-model-explorer, origin/feature/ai-model-explorer) Final commit - 30 Sept 2026
* 27eafe7 feat: integrate dynamic OpenRouter model explorer
* 3f6e98b feat: integrate dynamic OpenRouter model explorer
* c118a6c Deployed the application to minikube
* 55cf93b ci: add Jenkins CI/CD pipeline
* 95c804f security: harden production container
* 7bebab8 build: containerize AI model explorer
* 13ebb04 (origin/staging, origin/main, origin/HEAD, main) Initial commit - 29th Sept 2026
(.venv)
Balaje@Karthik MINGW64 /h/ai-model-explorer (feature/ai-model-explorer)
$

```

Figure 4 — GitFlow evidence: feature/ai-model-explorer is the active feature branch, with main and origin/staging visible, plus the project commit history and remote branch references.

3. Application Architecture

- Frontend: HTML/CSS/JavaScript served by Flask.
- Backend: Flask API layer in app.py.
- External catalog: OpenRouter /api/v1/models.
- Normalization: provider detection, model type inference, capabilities, pricing, launch date and modality normalization.
- Containerization: multi-stage python:3.10-slim image.
- CI/CD: Jenkins pipeline with linting, tests, Docker build, Trivy gate and registry push.
- Registry: local Docker Registry on localhost:5001.
- Orchestration: Minikube/Kubernetes Deployment and NodePort Service.
- Credentials: OPENROUTER_API_KEY supplied through a Kubernetes Secret, not embedded in source.

4. Flask API Evidence

Endpoint	Purpose
GET /	Serves the AI Model Explorer UI.
GET /health	Health check used by Kubernetes liveness/readiness probes.
GET /api/companies	Returns supported company/provider groups.
GET /api/companies/<company>/models	Returns normalized models for the selected company.
GET /api/models/<path:model_id>	Returns detailed metadata for a selected model.

5. Dynamic OpenRouter Explorer Evidence

- Company selection is dynamic and displays live model counts.
- Model versions come from OpenRouter rather than a static model catalog.

- Model Type is derived from the currently selected company's model data, so irrelevant global types are not shown.
- Details include company logo, model icon/initials, model ID, model type, launch date, context length, architecture, pricing, input/output modalities, capabilities and supported parameters.
- The captured Minikube screen shows Anthropic selected and Claude Opus 5.5 details.

6. Quality Gate Evidence

Command	Result	Observed Evidence
<code>python -m flake8 app.py tests</code>	PASS	No output / no lint errors.
<code>python -m black --check app.py tests</code>	PASS	2 files would be left unchanged.
<code>python -m pytest -v</code>	PASS	7 passed in 0.20s.

7. Docker Evidence

- Multi-stage Dockerfile using python:3.10-slim.
- Runtime image measured at approximately 128.1 MiB, below the 150 MB target.
- Trivy 0.74.0 used for vulnerability gating.
- Runtime setuptools and wheel metadata were removed to resolve the earlier base-image vulnerability findings.
- Final Trivy gate reported zero HIGH/CRITICAL vulnerabilities under the configured scan.

```
Dockerfile
5  ENV PYTHONDONTWRITEBYTECODE=1
6  ENV PYTHONUNBUFFERED=1
7
8  COPY requirements.txt .
9
10 RUN pip install --no-cache-dir \
11     --prefix=/install \
12     -r requirements.txt
13
14
15 FROM python:3.10-slim
16
17 WORKDIR /app
18
19 ENV PYTHONDONTWRITEBYTECODE=1
20 ENV PYTHONUNBUFFERED=1
21
22 COPY --from=builder /install /usr/local
23
24 RUN python -m pip uninstall -y setuptools wheel \
25     && rm -rf /root/.cache/pip
26
27 COPY app.py .
28 COPY templates ./templates
29 COPY static ./static
30
31 EXPOSE 5000
32
33 CMD ["python", "app.py"]
```

Figure 5 — Dockerfile evidence: multi-stage build with a Python 3.10 slim runtime, COPY --from=builder, runtime cleanup of setuptools/wheel, application files, exposed port 5000, and the Python application startup command.

```
(.venv) PS H:\ai-model-explorer> trivy image --severity HIGH,CRITICAL --ignore-unfixed localhost:5001/ai-model-explorer:8
```

Target	Type	Vulnerabilities	Secrets
localhost:5001/ai-model-explorer:8 (debian 13.7)	debian	0	-
usr/local/lib/python3.10/site-packages/blinker-1.9.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/certifi-2026.7.22.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/charset_normalizer-3.5.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/click-8.5.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/flask-3.1.3.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/idna-3.20.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/itsdangerous-2.2.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/jinja2-3.1.6.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/markupsafe-3.0.3.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/packaging-26.3.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/pip-23.0.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/requests-2.34.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/urllib3-2.8.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.10/site-packages/werkzeug-3.1.9.dist-info/METADATA	python-pkg	0	-

Legend:

- '-': Not scanned
- '0': Clean (no security findings detected)

Figure 6 — Trivy security evidence for localhost:5001/ai-model-explorer:8: 0 vulnerabilities reported for the Debian image and Python package metadata targets.

8. Jenkins CI/CD Evidence

- Jenkins 2.568.3 with Java 21.
- Job: ai-model-explorer-pipeline.
- SCM branch: feature/ai-model-explorer.
- Build #8 completed successfully.
- Pipeline stages: Checkout → Build → Docker Build.
- Build stage installs development dependencies and runs Black, Flake8 and Pytest.
- Docker Build stage builds the image, runs the Trivy HIGH/CRITICAL gate and pushes only after the gate passes.

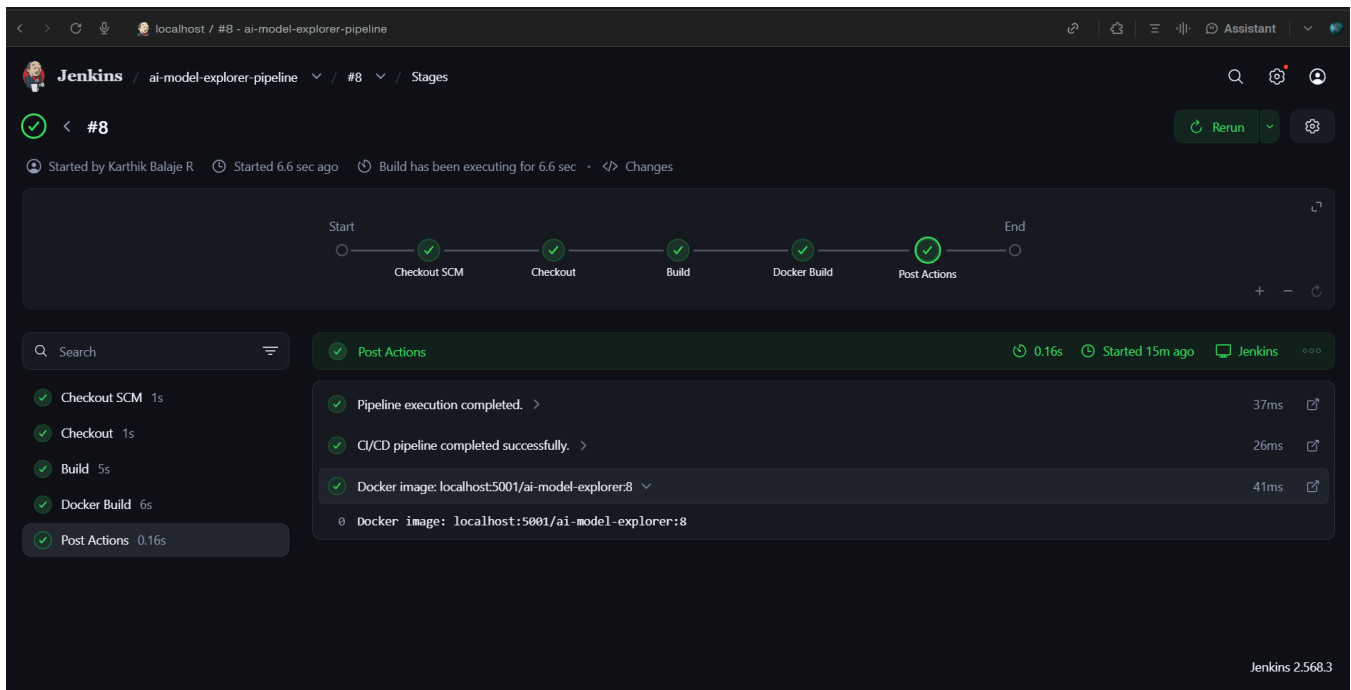


Figure 2 — Jenkins Build #8: successful Checkout, Build, Docker Build and Post Actions; Docker image localhost:5001/ai-model-explorer:8.

9. Registry Evidence

Verified registry query:

```
curl.exe http://127.0.0.1:5001/v2/ai-model-explorer/tags/list
```

Observed: {"name":"ai-model-explorer","tags":["8","6"]}

Build #8 is therefore present in the local registry alongside the previous tag 6.

10. Kubernetes / Minikube Evidence

- Minikube v1.39.0 using the Docker driver.
- Deployment: ai-model-explorer.
- Replicas: 2.
- Strategy: RollingUpdate with maxUnavailable: 0 and maxSurge: 1.
- Liveness probe: GET /health on port 5000.
- Readiness probe: GET /health on port 5000.
- CPU and memory requests/limits configured.
- Service type: NodePort, port 80 to targetPort 5000.
- Deployment YAML passed kubectl apply --dry-run=client.
- The application was successfully opened using the Minikube service URL.

```

• (.venv) PS H:\ai-model-explorer> kubectl get deployment ai-model-explorer
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
ai-model-explorer   2/2     2             2            13m
(.venv) PS H:\ai-model-explorer> kubectl get pods -l app=ai-model-explorer
• NAME                                READY   STATUS    RESTARTS   AGE
ai-model-explorer-6d57455ccb-72tq4   1/1     Running   0           13m
ai-model-explorer-6d57455ccb-kvq8j   1/1     Running   0           13m
(.venv) PS H:\ai-model-explorer> kubectl get service ai-model-explorer
• NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
ai-model-explorer    NodePort    10.103.137.93 <none>        80:31340/TCP     73m
❖ (.venv) PS H:\ai-model-explorer> minikube service ai-model-explorer --url
http://127.0.0.1:62987
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.

```

Figure 3 — Kubernetes/Minikube evidence: the *ai-model-explorer* Deployment shows 2/2 ready and available, both pods are Running, the NodePort Service is present, and the Minikube service command returns the local service URL.

11. OpenRouter Secret / Security Evidence

The Kubernetes deployment references an *openrouter-secret* Secret and reads *OPENROUTER_API_KEY* using *secretKeyRef*. The API key is not intended to be stored in *app.py*, *deployment.yaml*, the *Dockerfile* or *GitHub*.

12. Application Output Evidence

Figure 1 shows the AI Model Explorer running through the Minikube service URL. The screenshot demonstrates company selection, dynamic model type filtering and detailed live model metadata.

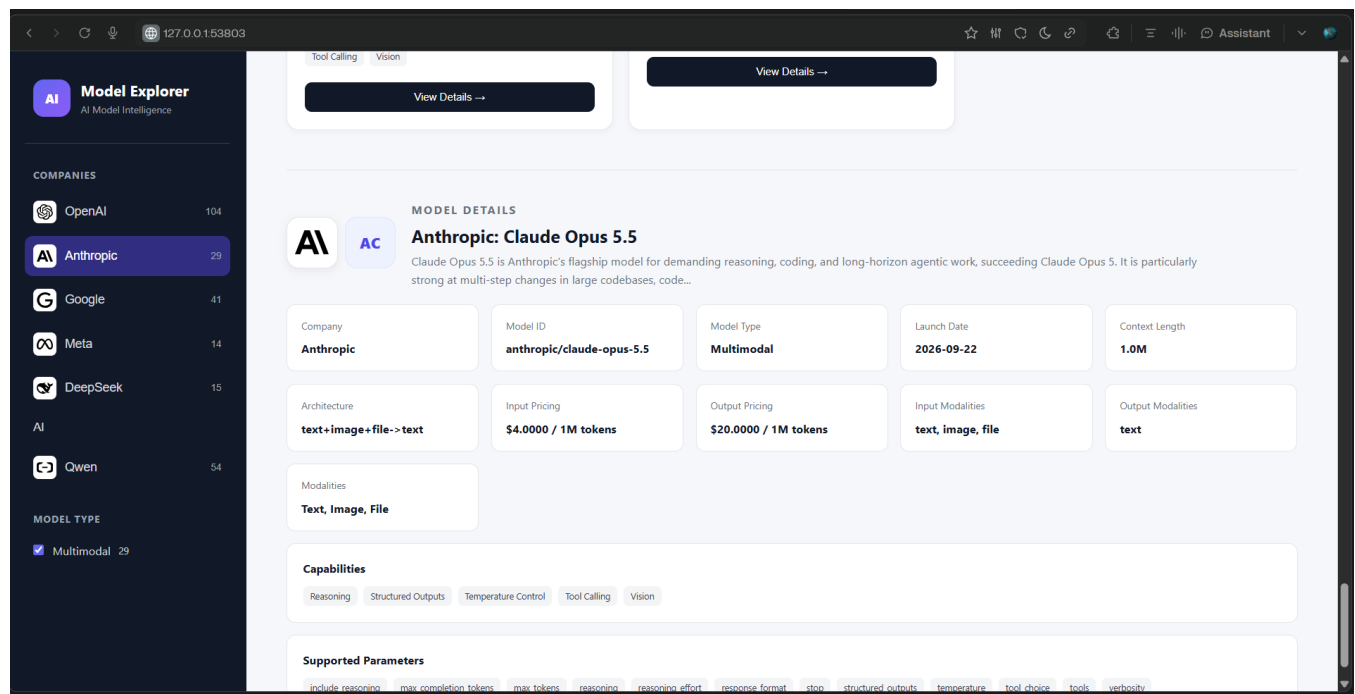


Figure 1 — AI Model Explorer running through Minikube; Anthropic / Claude Opus 5.5 selected.

13. Conclusion

Case Study 1 has been implemented as an end-to-end CI/CD and Kubernetes workflow around the AI Model Explorer. Local formatting, linting and automated tests passed; Jenkins Build #8 succeeded and pushed image tag 8 to the local registry; the Kubernetes application was launched through Minikube; and live application output was captured.

This report is ready to accompany the repository and CI/CD evidence. The final administrative step is to confirm the Git working tree is clean after committing the final deployment-manifest change.

This implementation was prepared covering CI/CD with GitHub and Jenkins, Docker containerization, Jenkins fundamentals and hands-on Kubernetes/Minikube.